DMI COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

(APPROVED BY AICTE, AFFILIATED TO ANNA UNIVERSITY,
ACCREDITED BY NBA, ISO CERTIFIED INSTITUTION)
PALANCHUR – NAZARATHPET P.O., CHENNAI – 600 123

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS1931 –CLOUD COMPUTING LABORATORY

NAME OF STUDENT : ……..

REGISTER NUMBER : ……..

YEAR / CLASS : III / CSE

SEMESTER : 05

ACADEMIC YEAR : 2026–2027

DMI COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

PALANCHUR – NAZARATHPET P.O., CHENNAI – 600 123

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

UNIVERSITY REGISTER NUMBER _____

Certified that this is the bonafide record of practical work done by Mr./Ms. .....of the **Department of Computer Science and Engineering** in the **..... Laboratory** during the **V Semester, Academic Year 2026–2027**, and submitted for the end semester Practical Examination conducted on ..... at **DMI College of Engineering, Palanchur, Chennai, TN – 600 123.**

SUBJECT IN-CHARGE **HEAD OF THE DEPARTMENT**

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

S.No	DATE	NAME OF THE EXPERIMENT	PAGE NO	SIGNATURE

Exp: 1	CREATING ARRAYS AND DISPLAYING ARRAY INFORMATION USING NUMPY

Aim:

To install NumPy in Python, create an array, and display its shape, dimensions, and data type.

Software Requirements:

- Ubuntu Operating System
- Python 3
- NumPy
- Ubuntu Terminal
- pip3

Algorithm / Procedure:

START
↓
Open Ubuntu Terminal
↓
Update Ubuntu packages
↓
Install/verify Python and pip
↓
Install NumPy
↓
Create a Python program
↓
Import NumPy
↓
Create a NumPy array
↓
Display the array
↓
Display shape, dimensions and data type
↓
Execute the program
↓
STOP

Step 1: Start Ubuntu

Open the Ubuntu virtual machine and log in.

Step 2: Open Terminal

Press:

Ctrl + Alt + T

Step 3: Update Ubuntu

Run:

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password: (no echo)
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
```

Enter the Ubuntu password if asked and wait until the update is completed.

Step 4: Upgrade Installed Packages

Run:

sudo apt upgrade -y

```
renuk@RENUKA:~$ sudo apt upgrade -y
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  python3-pip-whl python3-pip
```

Wait until the installation is completed.

Step 5: Check Python Installation

Run:

python3 --version

Example output:

Python 3.12.3

```
renuk@RENUKA:~$ python3 --version
Python 3.14.4
renuk@RENUKA:~$ pip3 --version
pip 25.1.1 from /usr/lib/python3/dist-packages/pip (python 3.14)
```

Step 6: Check pip Installation

Run:

pip3 –version

```
renuk@RENUKA:~$ pip3 --version
pip 25.1.1 from /usr/lib/python3/dist-packages/pip (python 3.14)
```

If pip is not installed, install it using:

sudo apt install python3-pip -y

Step 7: Install NumPy

Run:

pip3 install numpy

If the system requires the Ubuntu package method, use:

sudo apt install python3-numpy -y

```
renuk@RENUKA:~$ sudo apt install python3-numpy -y
Installing:
  python3-numpy
```

Step 8: Verify NumPy

Run:

python3

Then type:

import numpy as np

If no error appears, NumPy is installed successfully.

```
renuk@RENUKA:~$ python3
Python 3.14.4 (main, Aug 20 2026, 10:41:58) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np
>>> exit()
```

Exit Python using:

exit()

STEP 9:Create the first python program

Create a file named array1.py:

nano array1.py

Enter the following program:

```
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])
print(numbers)
```

Save the file using:

Ctrl + O → Enter → Ctrl + X

Run the program:

python3 array1.py

Output:

[10 20 30 40 50]

```
renuk@RENUKA:~$ nano array.py
renuk@RENUKA:~$ python3 array.py
[10 20 30 40 50]
```

STEP 10 : Create the Second python program

Create a file named array2.py:

nano array2.py

Enter the following program:

```
import numpy as np

matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix)
```

Save the file using:

Ctrl + O → Enter → Ctrl + X

Run the program:

python3 array2.py

```
renuk@RENUKA:~$ nano array.py
renuk@RENUKA:~$ python3 array.py
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Output:

[[1 2 3]

[4 5 6]

[7 8 9]]

STEP 11 : Create the third python program

Create a file named array3.py:

nano array3.py

Enter the following program:

```
import numpy as np
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(matrix)
print("Shape:", matrix.shape)
print("Dimensions:", matrix.ndim)
print("Data type:", matrix.dtype)
```

Save the file using:

Ctrl + O → Enter → Ctrl + X

Run the program:

python3 array3.py

[[1 2 3]

 [4 5 6]

 [7 8 9]]

Shape: (3, 3)

Dimensions: 2

Data type: int64

Output:

```
renuk@RENUKA:~$ nano array.py
renuk@RENUKA:~$ python3 array.py
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
Dimensions: 2
Data type: int64
```

RESULT:

Thus, NumPy was successfully installed and the three Python programs were successfully executed. The NumPy arrays were created and displayed, and the array shape, dimensions, and data type were successfully obtained.

Exp: 2	**Install a C Compiler in Ubuntu Virtual Machine (Oracle VM VirtualBox) and Execute Simple C Programs**

Aim:

To install the GCC C compiler in an Ubuntu virtual machine created usi ng Oracle VM VirtualBox and execute simple C programs.

Procedure:

Step 1: Open Oracle VM VirtualBox

1. Turn on your computer.
2. Open **Oracle VM VirtualBox**.
3. The **Oracle VM VirtualBox Manager** window appears.

Step 2: Start the Ubuntu Virtual Machine

1. In the left panel, select **Ubuntu**.
2. Click the **Start** (green arrow) button.
3. Wait for Ubuntu to boot.
4. Log in using your Ubuntu username and password.

You are now inside the Ubuntu Virtual Machine.

Step 3: Open the Terminal in Ubuntu

There are two methods:

Method 1

- Press **Ctrl + Alt + T**

Method 2

- Click **Show Applications**
- Search for **Terminal**
- Click **Terminal**

The Terminal window will open.

Step 4: Update Ubuntu Packages

In the Terminal, type:

sudo apt update

Press **Enter**.

If prompted, enter your Ubuntu password and press **Enter**.

Wait until the update process completes

Step 5: Install the GCC C Compiler

Type:

sudo apt install gcc

Press **Enter**.

When prompted:

Do you want to continue? [Y/n]

Type:

Y

and press **Enter**.

Wait for the installation to finish.

Step 6: Verify the Installation

Type:

gcc --version

Press **Enter**.

If GCC is installed successfully, the version information will be displayed.

Step 7: Create a C Program

Type:

gedit hello.c

or, if gedit is not available:

nano hello.c

A text editor opens.

Type the following program:

```
#include <stdio.h>
int main()
{
   printf("Hello, World!\n");
   return 0;
}
```

Save the file as **hello.c**.

Step 8: Compile the Program

Return to the Terminal and type:

gcc hello.c -o hello

Press **Enter**.

If there are no errors, the program compiles successfully.

Step 9: Execute the Program

Type:

./hello

Press **Enter**.

Output:

Hello, World!

Execute Another Simple Program

Step 10: Create Another Program

Type:

gedit add.c

or

nano add.c

Enter the following program:

```
#include <stdio.h>
int main()
{
   int a, b;
   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);
   printf("Sum = %d\n", a + b);
   return 0;
}
```

Save the file.

Step 11: Compile the Program

gcc add.c -o add

Step 12: Execute the Program

./add

Sample Output

Enter two numbers: 10 20

Sum = 30

Output:

- GCC compiler is installed successfully in the Ubuntu Virtual Machine.
- The **Hello World** program executes and displays:

Hello, World!

- The **Addition** program executes and displays the correct sum of two numbers.

Screen shot 1:

```
lab6-pc24@lab6pc24-OptiPlex-7040:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu focal InRelease [57.7 kB]
Hit:2 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:3 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu focal InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:6 http://in.archive.ubuntu.com/ubuntu focal-backports InRelease
Fetched 57.7 kB in 1s (41.5 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
W: Target Packages (stable/binary-amd64/Packages) is configured multiple times in /etc/apt/sources.lis
etc/apt/sources.list.d/docker.list:1
```

Screen shot 2:

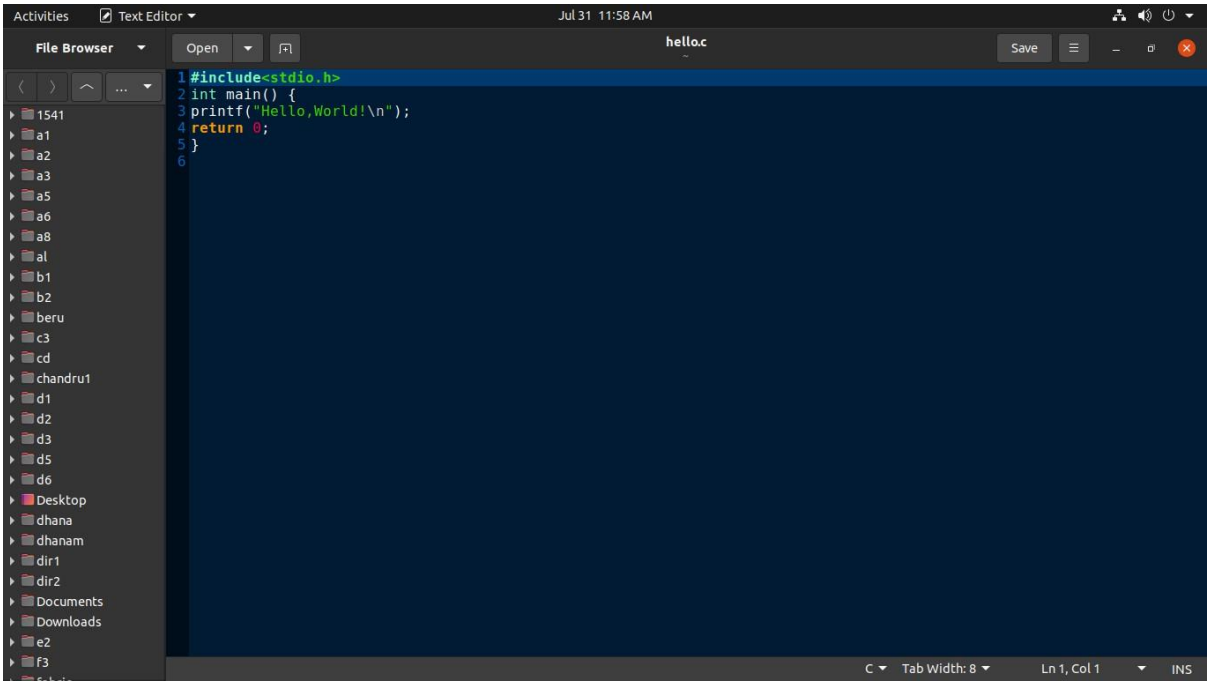
```
urces.list.d/docker.list:1
lab6-pc24@lab6pc24-OptiPlex-7040:~$ sudo apt install gcc
Reading package lists... Done
Building dependency tree
Reading state information... Done
gcc is already the newest version (4:9.3.0-1ubuntu2).
gcc set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gcc --version
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

lab6-pc24@lab6pc24-OptiPlex-7040:~$ gedit hello.c



^C
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gcc hello.c
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gcc hello.c -o hello
lab6-pc24@lab6pc24-OptiPlex-7040:~$ ./hello
Hello,World!
lab6-pc24@lab6pc24-OptiPlex-7040:~$
```

Screen shot 3:



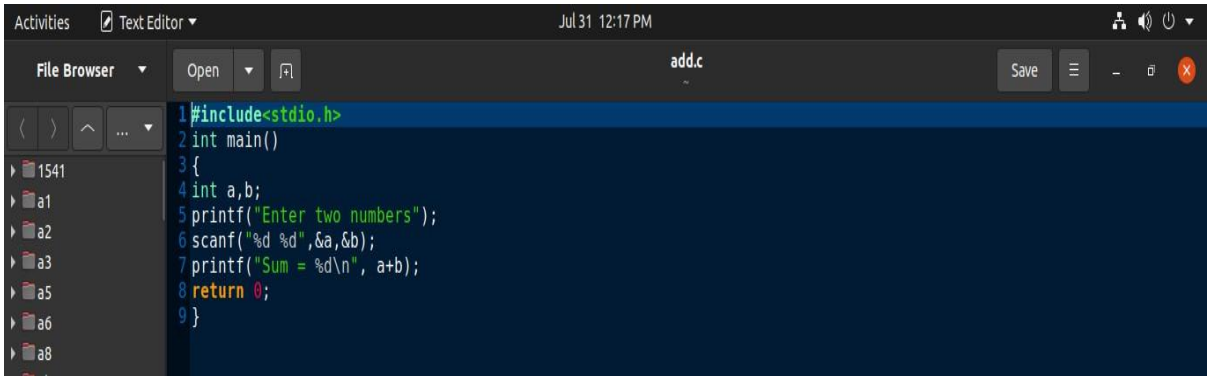
Screen shot 4:

```
lab6-pc24@lab6pc24-OptiPlex-7040:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu focal InRelease [57.7 kB]
Hit:2 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:3 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu focal InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:6 http://in.archive.ubuntu.com/ubuntu focal-backports InRelease
Fetched 57.7 kB in 1s (41.5 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
W: Target Packages (stable/binary-amd64/Packages) is configured multiple times in /etc/apt/sources.lis
etc/apt/sources.list.d/docker.list:1
```

Screen shot 5:

```
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gedit hello.c
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gedit add.c
^C
lab6-pc24@lab6pc24-OptiPlex-7040:~$ gcc add.c -o add
lab6-pc24@lab6pc24-OptiPlex-7040:~$ ./add
Enter two numbers 10 20
Sum = 30
lab6-pc24@lab6pc24-OptiPlex-7040:~$
```

Screen shot 6:



Result:

The GCC C compiler was successfully installed in the Ubuntu Virtual Machine running in Oracle VM VirtualBox, and simple C programs were compiled and executed successfully.

Exp: 3	Installation of Google App Engine and Development of Simple Web Applications Using Python on Ubuntu

Aim:

To install the Google Cloud SDK (App Engine tools) on Ubuntu and develop a simple Python web application ("Hello World"), then deploy it to Google App Engine.

Requirements:

- Ubuntu 20.04/22.04
- Python 3.8 or later
- Internet connection
- Google Account
- Google Cloud SDK
- Text Editor (VS Code or Gedit)
- Terminal

Theory:

Google App Engine (GAE) is a Platform as a Service (PaaS) provided by Google Cloud. It enables developers to build, deploy, and scale web applications without managing servers.

Features:

- Automatic scaling
- Load balancing
- Secure environment
- Supports Python, Java, Go, Node.js, PHP, and Ruby
- Easy deployment using Cloud SDK

Procedure:

Step 1: Update Ubuntu

Open Terminal and execute:

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password:
```

sudo apt upgrade -y

```
renuk@RENUKA:~/CloudSimLab$ sudo apt install google-cloud-cli -y
Installing:
  google-cloud-cli
```

Step 2: Install Python

Check whether Python is installed.

python3 --version

If not installed,

sudo apt install python3 python3-pip

-y Check pip version

pip3 –version

```
renuk@RENUKA:~/CloudSimLab$ python3 --version
Python 3.14.4
renuk@RENUKA:~/CloudSimLab$ pip3 --version
Command 'pip3' not found, but can be installed with:
sudo apt install python3-pip
renuk@RENUKA:~/CloudSimLab$ sudo apt install python3-pip
[sudo: authenticate] Password:
Installing:
  python3-pip
```

Step 3: Install Google Cloud SDK

Install required packages

sudo apt install apt-transport-https ca-certificates gnupg curl -y

```
renuk@RENUKA:~/CloudSimLab$ sudo apt install apt-transport-https ca-certificates gnupg curl -y
ca-certificates is already the newest version (20260601~26.04.1).
ca-certificates set to manually installed.
```

Add Google's repository key

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add –

```
renuk@RENUKA:~/CloudSimLab$ curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   1022 100   1022    0     0   3206      0                              0
```

Add repository

echo "deb https://packages.cloud.google.com/apt cloud-sdk main" | sudo tee /etc/apt/sources.list.d/google-cloud-sdk.list

```
renuk@RENUKA:~/CloudSimLab$ echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://packages.cloud.google.com/
/apt/sources.list.d/google-cloud-sdk.list
deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://packages.cloud.google.com/apt cloud-sdk main
```

Update package list

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password:
```

Install Cloud SDK

sudo apt install google-cloud-cli -y

```
renuk@RENUKA:~/CloudSimLab$ sudo apt install google-cloud-cli -y
Installing:
  google-cloud-cli
```

Verify

installation

gcloud version

```
renuk@RENUKA:~/CloudSimLab$ gcloud --version
Google Cloud SDK 585.0.0
bq 2.1.38
core 2026.09.11
gcloud-crc32c 1.0.0
gsutil 5.37
```

Step 4: Initialize Google Cloud SDK

Run

gcloud

init

```
renuk@RENUKA:~/CloudSimLab$ gcloud init
Welcome! This command will take you through the configuration of gcloud.
```

Follow the instructions:

- Login using your Google account.
- Create or select an existing project.
- Select the project.

Example

Choose

account Select

Project

Configuration Complete

Step 5: Install App Engine Component

gcloud components install app-engine-python

Initialize App Engine

gcloud app create

```
renuk@RENUKA:~/CloudSimLab$ gcloud config set project renuka-gae-lab-2026
Updated property [core/project].
renuk@RENUKA:~/CloudSimLab$ gcloud config get-value project
renuka-gae-lab-2026
renuk@RENUKA:~/CloudSimLab$ gcloud app create
You are creating an app for project [renuka-gae-lab-2026].
```

Choose the region.

Example:

asia-south1

Program 1: Hello World

Application Step 1: Create Project

Folder

mkdir

HelloApp cd

HelloApp

Step 2: Install Flask

pip3 install flask gunicorn

Step 3: Create app.py

from flask import Flask

```
renuk@RENUKA:~/CloudSimLab$ mkdir -p ~/Exp3
cd ~/Exp3
renuk@RENUKA:~/Exp3$ pwd
/home/renuk/Exp3
renuk@RENUKA:~/Exp3$ nano main.py
renuk@RENUKA:~/Exp3$ from flask import Flask
```

app = Flask(__name__)
@app.route("/")

def hello():

  return "Hello World from Google App Engine!"
if __name__ == "__main__":

  app.run(host="127.0.0.1", port=8080, debug=True)

Step 4: Create app.yaml

runtime: python311
entrypoint: gunicorn -b :$PORT app:app

Step 5: Test Locally

Run

python3 app.py

Open browser

http://127.0.0.1:8080

```
(venv) renuk@RENUKA:~/Exp3$ python main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
```

Output:

Hello World from Google App Engine!

Hello World from Google App Engine!

Step 6: Deploy Application

gcloud app deploy

Confirm

deployment Y

Step 7: Open Application

gcloud app browse

or

gcloud app browse --project=PROJECT_ID

Program 2: Simple Addition Web

Application app.py

```
from flask import Flask, request
app = Flask(__name__)
@app.route("/")
def home():
   return '''
   <h2>Addition Program</h2>
   <form action="/add">
   Number1:<input type="number"
   name="a"><br><br> Number2:<input
   type="number" name="b"><br><br>
   <input type="submit" value="Add">
   </form
   > '''
@app.route("/add")
def add():
   a = int(request.args.get("a"))
   b =
   int(request.args.get("b"))
   return f"Sum = {a+b}"
```

```
if __name__== "__main__
   ": app.run()
```

Deploy using

gcloud app

deploy Output

Enter two numbers

10
20
Sum = 30



Program 3: Student Information Application

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def student():
   return """
   <h2>Student Details</h2>
   Name : John <br>
   Department : CSE <br>
   College : DMI College of
   Engineering """

   if __name__== "__main__": app.run()
```

Output:

Student Details
Name : John

Department : CSE

College : DMI College of Engineering



Program 4: Current Date and Time

```
from flask import Flask

import datetime

app = Flask(__name__)
@app.route("/")

def time():

   now = datetime.datetime.now()

   return str(now)

if __name__== "__main__

   ": app.run()
```

Output:

2026-08-07 11:45:10



Result:
The Google App Engine environment was successfully installed and simple web applications were developed using Python and Flask. The applications were executed successfully and their outputs were obtained through the web browser.

Exp: 4	Launch Web Applications Using Google App Engine

Aim:

To use the **Google App Engine (GAE) Launcher** to create, configure, and launch a simple web application in Ubuntu.

Software Requirements

- Ubuntu Linux
- Python
- Google App Engine SDK / Google Cloud SDK
- Google App Engine Launcher
- Web browser
- Terminal

Note: The old standalone **Google App Engine Launcher** was part of the legacy App Engine SDK and is no longer the standard tool in current Google Cloud tooling. For a modern Ubuntu lab, the equivalent workflow is done using the **Google Cloud CLI (gcloud)**. If your university lab manual specifically says "GAE Launcher," the experiment can still be written using the same objective and procedure, while using gcloud to launch the application.

Algorithm / Procedure:

START

↓

Install Google Cloud CLI

↓

Login to Google Cloud

↓

Create Google Cloud Project

↓

Create Python Web Application

↓

Create app.yaml

↓

Test Application Locally

↓

Create App Engine Application

↓

Deploy using gcloud app deploy

↓

Launch using gcloud app browse

↓

Display Web Application

↓

STOP

1. Install Google Cloud CLI

Open Terminal:

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password:
```

Install required packages:

sudo apt install apt-transport-https ca-certificates gnupg curl -y

```
renuk@RENUKA:~/CloudSimLab$ sudo apt install apt-transport-https ca-certificates gnupg curl -y
ca-certificates is already the newest version (20260601~26.04.1).
ca-certificates set to manually installed.
```

Add the Google Cloud package repository:

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \

```
renuk@RENUKA:~/CloudSimLab$ curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   1022 100   1022    0     0   3206      0                              0
```

sudo gpg --dearmor -o

/usr/share/keyrings/cloud.google.gpg Add the repository:

echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg]
https://packages.cloud.google.com/apt cloud-sdk main" | \

```
renuk@RENUKA:~/CloudSimLab$ echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://packages.cloud.google.com/
/apt/sources.list.d/google-cloud-sdk.list
deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://packages.cloud.google.com/apt cloud-sdk main
```

sudo tee /etc/apt/sources.list.d/google-cloud-

sdk.list Install Google Cloud CLI:

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password:
```

sudo apt install google-cloud-cli -y

```
renuk@RENUKA:~/CloudSimLab$ sudo apt install google-cloud-cli -y
Installing:
  google-cloud-cli
```

Check the installation:

gcloud version

```
renuk@RENUKA:~/CloudSimLab$ gcloud --version
Google Cloud SDK 585.0.0
bq 2.1.38
core 2026.09.11
gcloud-crc32c 1.0.0
gsutil 5.37
```

2. Login to Google Cloud

Run:

gcloud auth login

A browser window will open.

Login using your Google

account.

Then check the authenticated account:

gcloud auth list

```
(venv) renuk@RENUKA:~/gae-lab$ gcloud auth list
           Credentialed Accounts
ACTIVE  ACCOUNT
*       renukadevi2255t@gmail.com

To set the active account, run:
    $ gcloud config set account `ACCOUNT`
```

3. Create a Google Cloud Project

Create a project from the Google Cloud Console or

use: gcloud projects create my-gae-lab-project

Set the project:

gcloud config set project my-gae-lab-project

```
(venv) renuk@RENUKA:~/gae-lab$ gcloud config get-value project
renuka-gae-lab-2026
```

Replace my-gae-lab-project with your own unique Google Cloud project ID.

4.Create a Simple Web Application

Create a project directory:

mkdir ~/gae-lab

cd ~/gae-lab

Create the Python

application: nano main.py

Enter:

```
from flask import Flask
app = Flask(__name__)
@app.route('/')

def hello():

    return 'Hello World! Welcome to Google App Engine.'
if __name__ == '__main__':

    app.run(host='127.0.0.1', port=8080,

    debug=True)
```

Save the file.

```
(venv) renuk@RENUKA:~/Exp3$ cd ~
mkdir -p ~/gae-lab
cd ~/gae-lab
pwd
/home/renuk/gae-lab
(venv) renuk@RENUKA:~/gae-lab$ nano main.py
(venv) renuk@RENUKA:~/gae-lab$ ls
main.py
(venv) renuk@RENUKA:~/gae-lab$ nano requirements.txt
(venv) renuk@RENUKA:~/gae-lab$ ls
main.py  requirements.txt
(venv) renuk@RENUKA:~/gae-lab$ nano app.yaml
(venv) renuk@RENUKA:~/gae-lab$ ls
app.yaml  main.py  requirements.txt
(venv) renuk@RENUKA:~/gae-lab$ python3 -m pip install -r requirements.txt
Collecting Flask==3.0.0 (from -r requirements.txt (line 1))
```

4. Create requirements.txt

nano

requirements.txt

Enter:

Flask==3.0.0

Save the file.

5. Create app.yaml

Create the App Engine configuration

file: nano app.yaml

Enter:

runtime: python312

Save the file.

```
(venv) renuk@RENUKA:~/gae-lab$ nano app.yaml
(venv) renuk@RENUKA:~/gae-lab$ ls
app.yaml  main.py  requirements.txt
```

The directory should now

contain: gae-lab/

│

├── main.py

├── app.yaml

└── requirements.txt

Check:

ls

6. Test the Web Application Locally

Install Flask:

pip3 install -r

requirements.txt Run:

python3 main.py

You should see:

Running on

http://127.0.0.1:8080 Open the

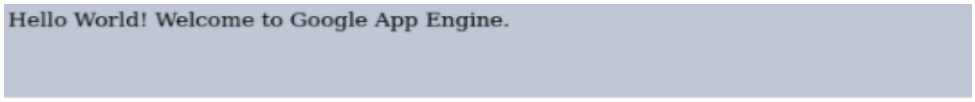
browser and enter:

http://127.0.0.1:8080

```
(venv) renuk@RENUKA:~/gae-lab$ cd ~/gae-lab
ls
app.yaml  main.py  requirements.txt
(venv) renuk@RENUKA:~/gae-lab$ source ~/Exp3/venv/bin/activate
python main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:8080
```

The browser should display:

Hello World! Welcome to Google App Engine.



Press:

Ctrl + C

to stop the local server.

7. Create an App Engine Application

Before deployment, select the appropriate

region: gcloud app create

Follow the prompt and select the required region.

8. Launch the Web Application

From the application directory:

cd ~/gae-lab

Deploy the application:

gcloud app deploy

When prompted:

Do you want to continue

(Y/n)? Enter:

Y

After successful deployment, launch the application:

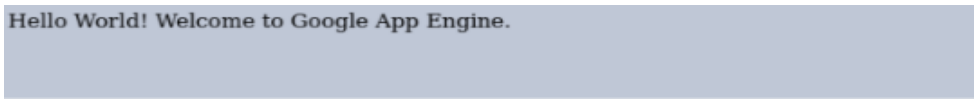
gcloud app browse

The web application will open in your default browser.

9. Expected Output

The browser displays:

Hello World! Welcome to Google App Engine.



The application is now running on **Google App Engine**.

Result:

The web application was successfully created and deployed using **Google App Engine**. The application was launched successfully and accessed through a web browser.

Exp: 5	Cloud Simulation and Implementation of a New Scheduling Algorithm Using

Aim:

To simulate a cloud computing environment using **CloudSim** and implement a **Min-Min scheduling algorithm** for scheduling cloudlets to virtual machines.

Software Requirements:

- Ubuntu Linux
- Java JDK
- CloudSim 3.0.3
- Terminal
- Text editor / VS Code / Eclipse

Algorithm: Min-Min

Scheduling Step 1

Create a set of cloudlets and virtual machines.

Step 2

Calculate the expected completion time of every cloudlet on every available VM.

Step 3

For each cloudlet, identify the VM that provides the minimum completion time.

Step 4

Among those minimum completion times, select the **globally minimum** one.

Step 5

Assign that cloudlet to the selected VM.

Step 6

Update the availability time of the selected VM.

Step 7

Repeat until all cloudlets are scheduled.

1. Install Java in Ubuntu

Open Terminal:

sudo apt update

```
renuk@RENUKA:~$ sudo apt update
[sudo: authenticate] Password:
```

sudo apt install openjdk-8-jdk -y

```
renuk@RENUKA:~$ sudo apt install openjdk-8-jdk -y
Installing:
  openjdk-8-jdk
```

Check installation:

java -version

javac -version

```
renuk@RENUKA:~$ java -version
openjdk version "1.8.0_502"
OpenJDK Runtime Environment (build 1.8.0_502-8u502-ga~us1-0ubuntu1~26.04-b07)
OpenJDK 64-Bit Server VM (build 25.502-b07, mixed mode)
renuk@RENUKA:~$ javac -version
javac 1.8.0_502
```

2. Download CloudSim

Download **CloudSim 3.0.3** and extract

it. For example:

cd ~/Downloads

unzip CloudSim-3.0.3.zip

mv CloudSim-3.0.3

~/cloudsim Check:

ls ~/cloudsim

You should see folders such as:

```
renuk@RENUKA:~$ ls /mnt/c/Users/renuk/Downloads/cloudsim-3.0.3
cloudsim-3.0.3
```

example

s jars

sources

3. Create the Lab Project

mkdir -p

~/CloudSimLab/src cd

~/CloudSimLab

Copy the CloudSim JAR:

cp ~/cloudsim/jars/cloudsim-3.0.3.jar

. Check:

ls

Expected:

cloudsim-

3.0.3.jar src

4. Create Java Program

Create the file:

nano src/MinMinCloudSim.java

For a lab experiment, the program should contain these major modules:

```
renuk@RENUKA:~$ mkdir -p ~/CloudSimLab/src
renuk@RENUKA:~$ cd ~/CloudSimLab
renuk@RENUKA:~/CloudSimLab$ pwd
/home/renuk/CloudSimLab
renuk@RENUKA:~/CloudSimLab$ cp /mnt/c/Users/renuk/Downloads/cloudsim-3.0.3/cloudsim-3.0.3/jars/cloudsim-3.0.3.jar .
renuk@RENUKA:~/CloudSimLab$ ls
cloudsim-3.0.3.jar  src
renuk@RENUKA:~/CloudSimLab$ nano src/MinMinCloudSim.java
renuk@RENUKA:~/CloudSimLab$ ls src
MinMinCloudSim.java
```

Initialize CloudSim

↓

Create Datacenter

↓

Create Broker

↓

Create Virtual Machines

↓

Create Cloudlets

↓

Apply Min-Min Scheduling

↓

Submit Cloudlets

↓

Start CloudSim

↓

Display Results

Min-Min algorithm

Suppose the cloudlets have execution lengths:

Cloudlet 0 → 40000

Cloudlet 1 → 10000

Cloudlet 2 → 30000

Cloudlet 3 → 20000

Cloudlet 4 → 50000

The algorithm selects the cloudlet with the **minimum completion time**

first. A simplified scheduling sequence could therefore be:

Cloudlet 1 → 10000

Cloudlet 3 → 20000

Cloudlet 2 → 30000

Cloudlet 0 → 40000

Cloudlet 4 → 50000

The important difference is that **Min-Min considers both the cloudlet and the available VM**, rather than simply sorting cloudlets by length.

5. Compile the

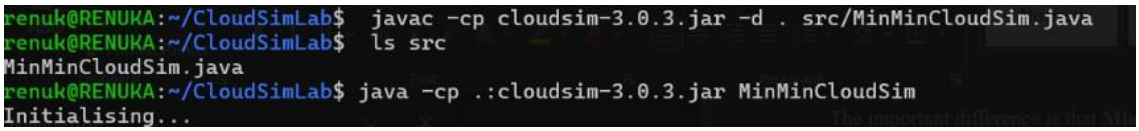
Program After saving

the Java file: cd

~/CloudSimLab

Compile:

javac -cp cloudsim-3.0.3.jar -d . src/MinMinCloudSim.java



If there is no error, the program has compiled successfully.

6. Run the CloudSim Experiment

java -cp .:cloudsim-3.0.3.jar

MinMinCloudSim You should obtain output

similar to:

========== MIN-MIN SCHEDULING ==========

Cloudlet 1 assigned to VM 0

Cloudlet 3 assigned to VM 1

Cloudlet 2 assigned to VM 0

Cloudlet 0 assigned to VM 1

Cloudlet 4 assigned to VM 0

========== SIMULATION RESULTS ==========

Cloudlet ID	Status	VM ID	Start Time	Finish Time
1	SUCCESS	0	0.0	10.0
3	SUCCESS	1	0.0	20.0
2	SUCCESS	0	10.0	40.0
0	SUCCESS	1	20.0	60.0
4	SUCCESS	0	40.0	90.0

==

The exact times and VM assignments will depend on the MIPS, number of PEs, cloudlet lengths, and scheduling implementation used in your program.

Output:

```
========== MIN-MIN SCHEDULING ==========
Cloudlet 1 assigned to VM 0
Cloudlet 3 assigned to VM 1
Cloudlet 2 assigned to VM 0
Cloudlet 0 assigned to VM 1
Cloudlet 4 assigned to VM 0
Starting CloudSim version 3.0
Datacenter_0 is starting...
Broker is starting...
Entities started.
0.0: Broker: Cloud Resource List received with 1 resource(s)
0.0: Broker: Trying to Create VM #0 in Datacenter_0
0.0: Broker: Trying to Create VM #1 in Datacenter_0
[VmScheduler.vmCreate] Allocation of VM #1 to Host #0 failed by MIPS
0.1: Broker: VM #0 has been created in Datacenter #2, Host #0
0.1: Broker: Creation of VM #1 failed in Datacenter #2
0.1: Broker: Postponing execution of cloudlet 0: bount VM not available
0.1: Broker: Sending cloudlet 1 to VM #0
0.1: Broker: Sending cloudlet 2 to VM #0
0.1: Broker: Postponing execution of cloudlet 3: bount VM not available
0.1: Broker: Sending cloudlet 4 to VM #0
10.1: Broker: Cloudlet 1 received
40.1: Broker: Cloudlet 2 received
90.1: Broker: Cloudlet 4 received
90.1: Broker: Destroying VM #0
90.1: Broker: Trying to Create VM #1 in CloudSimShutdown
Simulation: No more future events
CloudInformationService: Notify all CloudSim entities for shutting down.
Datacenter_0 is shutting down...
Broker is shutting down...
Simulation completed.
Simulation completed.
========== SIMULATION RESULTS ==========
Cloudlet ID     Status  VM ID   Start Time      Finish Time
1               SUCCESS 0       0.1             10.1
2               SUCCESS 0       10.1            40.1
4               SUCCESS 0       40.1            90.1
=========================================
```

Result:

The cloud computing environment was successfully simulated using CloudSim. The Min-Min scheduling algorithm was implemented as a custom scheduling approach and used to assign cloudlets to virtual machines based on minimum expected completion time.

Exp: 6	Transfer Files from One Virtual Machine to Another Virtual

Aim:

To transfer files from one virtual machine (VM) to another virtual machine using **SCP (Secure Copy Protocol)** in Ubuntu.

Software Requirements:

- Ubuntu Linux
- Two Virtual Machines
- VirtualBox / VMware
- OpenSSH
- Terminal
- Network connection between the VMs

Algorithm / Procedure

START

↓

Create VM1 and VM2

↓

Configure network between VMs

↓

Find IP addresses of both VMs

↓

Install and start SSH server on VM2

↓

Create a file in VM1

↓

Use SCP command

↓

Transfer file from VM1 to VM2

↓

Login to VM2

↓

Verify the transferred file

↓

STOP

1. Create Two Virtual Machines

Create two Ubuntu virtual machines:

VM1 – Source VM

VM2 – Destination VM

Example:

VM1 IP Address → 192.168.56.101

VM2 IP Address → 192.168.56.102

The IP addresses will be different in your system.

2. Check the IP Address of VM1

Open Terminal in

VM1: ip addr

or:

hostname -I

Example

output:

192.168.56.101

```
(venv) renuk@RENUKA:~/CloudSimLab$ hostname -I
172.17.38.92
```

3. Check the IP Address of VM2

Open Terminal in **VM2**:

hostname -I

Example:

192.168.56.102

```
(venv) renuk@RENUKA:~/CloudSimLab$ hostname -I
172.17.38.92
```

Make sure both VMs can communicate with each

other. Test from VM1:

ping 192.168.56.102

If the connection is successful, you will

see: 64 bytes from 192.168.56.102

Press:

Ctrl + C

to stop the ping.

4. Install OpenSSH Server on VM2

On **VM2**, execute:

sudo apt update

sudo apt install openssh-server -

y Check the SSH service:

sudo systemctl status

ssh If it is not running,

start it:

sudo systemctl start ssh

Enable it at startup:

sudo systemctl enable ssh

```
(venv) renuk@RENUKA:~/CloudSimLab$ sudo apt update
sudo apt install openssh-server -y
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:3 https://packages.cloud.google.com/apt cloud-sdk InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [478 kB]
Fetched 615 kB in 5s (134 kB/s)
60 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  openssh-client

Installing:
  openssh-server
```

5. Create a File in VM1

On **VM1**, create a test file:

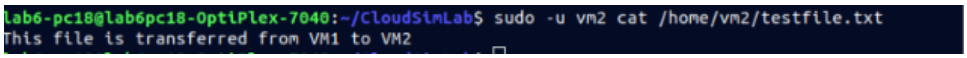
cd ~

echo "This file is transferred from VM1 to VM2" > testfile.txt
Check the file:

cat testfile.txt

Output:

This file is transferred from VM1 to VM2



6. Transfer the File from VM1 to VM2

Use the following command on **VM1**:

scp testfile.txt username@192.168.56.102:/home/username/



Replace:

- username → username of VM2
- 192.168.56.102 → IP address of

VM2 For example:

scp testfile.txt student@192.168.56.102:/home/student/

The first time you connect, Ubuntu may ask you to confirm the

host: Are you sure you want to continue connecting (yes/no)?

Enter:

yes

Then enter the password of the VM2 user.

The file will be transferred to VM2. scp supports the user@host:path format for specifying the destination.

7. Verify the File in

VM2 Login to **VM2** and

execute: ls

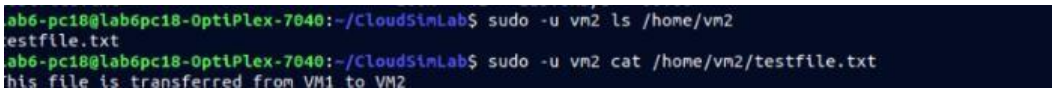
You should see:
testfile.txt

Display its contents:

cat testfile.txt

Output:

This file is transferred from VM1 to VM2



Therefore, the file has been successfully transferred.

8. Transfer a Directory

To transfer an entire directory, use the -r option.

Create a directory in VM1:

mkdir mydata

Create files:

echo "File 1" > mydata/file1.txt echo "File 2" > mydata/file2.txt Transfer the directory:

scp -r mydata student@192.168.56.102:/home/student/

The -r option copies directories recursively, including their contents.

SCP Syntax

Copy a file from VM1 to VM2

scp <filename> <username>@<VM2-IP>:<destination> Example:

scp testfile.txt student@192.168.56.102:/home/student/

```
lab6-pc18@lab6pc18-OptiPlex-7040:~/CloudSimLab$ scp testfile.txt vm2@192.168.3.23:/home/vm2/
The authenticity of host '192.168.3.23 (192.168.3.23)' can't be established.
ECDSA key fingerprint is SHA256:8hjdUuF4UMSsvlCUUWHjruwefkPHCrB7Q8LiUQYcXkg.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.3.23' (ECDSA) to the list of known hosts.
vm2@192.168.3.23's password:
testfile.txt                          100%   41   113.6KB/s   00:00
```

Copy a director

scp -r <directory> <username>@<VM2-IP>:<destination>

Example:
scp -r mydata student@192.168.56.102:/home/student/

Expected

Output : VM1

$ ls

testfile.txt

```
ab6-pc18@lab6pc18-OptiPlex-7040:~/CloudSimLab$ sudo -u vm2 ls /home/vm2
estfile.txt
ab6-pc18@lab6pc18-OptiPlex-7040:~/CloudSimLab$ sudo -u vm2 cat /home/vm2/testfile.txt
his file is transferred from VM1 to VM2
```

$ cat testfile.txt

This file is transferred from VM1 to VM2

VM2

$ ls

testfile.txt

$ cat testfile.txt

This file is transferred from VM1 to VM2

```
lab6-pc18@lab6pc18-OptiPlex-7040:~/CloudSimLab$ cat testfile.txt
This file is transferred from VM1 to VM2
```

Result:
The file was successfully transferred from one virtual machine to another virtual machine using **SCP over SSH** in the Ubuntu environment.

Exp: 7	Install Hadoop Single Node Cluster and Run Word Count

Aim

To install and configure Hadoop 3.3.6 Single Node Cluster on Ubuntu and execute a simple Word Count MapReduce application using HDFS and YARN.

Software Requirements

- Ubuntu Linux
- Java OpenJDK 11
- Hadoop Hadoop 3.3.6
- SSH OpenSSH
- Ubuntu Terminal

Algorithm / Procedure

START

↓

Install Java and Hadoop

↓

Configure Hadoop files

↓

Configure SSH

↓

Format Name Node

↓

Start HDFS and YARN

↓

Verify Hadoop services

↓

Create Word Count input file

↓

Upload file to HDFS

↓

Run Word Count MapReduce program

↓

Display the output

↓

STOP

Step 1: Open Ubuntu Terminal

Open the Ubuntu Terminal by pressing Ctrl + Alt + T.

Step 2: Check Java Installation

Command:

java -version

Expected output:

openjdk version "11.0.27"

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ java -version
openjdk version "11.0.27" 2025-04-15
```

Step 3: Check Hadoop Installation

Command:

hadoop version

Expected output:

Hadoop 3.3.6

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hadoop version
Hadoop 3.3.6
```

hadoop: command not found

Step 4: Open .bashrc

nano ~/.bashrc

Add these lines at the end:

export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

export HADOOP_HOME=/home/lab6-pc18/hadoop/hadoop-3.3.6

export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

export HADOOP_COMMON_HOME=$HADOOP_HOME

export HADOOP_HDFS_HOME=$HADOOP_HOME

export HADOOP_MAPRED_HOME=$HADOOP_HOME

export HADOOP_YARN_HOME=$HADOOP_HOME

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

Important: If your Ubuntu username is different from lab6-pc18, replace that username in the HADOOP_HOME path.

Save:

Ctrl + O

Enter

Ctrl + X

HDFS CONFIGURATION

Step 5: Configure core-site.xml

nano $HADOOP_HOME/etc/hadoop/core-site.xml

Enter:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Save and exit.

Step 6: Verify core-site.xml

hdfs getconf -confKey

fs.defaultFS Expected:

hdfs://localhost:9000

Step 7 :Configure hdfs-site.xml

nano $HADOOP_HOME/etc/hadoop/hdfs-

site.xml Enter:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/lab6-pc18/hadoopdata/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/lab6-pc18/hadoopdata/datanode</value>
  </property>
</configuration>
```

Save and exit.

Step 8: Create NameNode directory

mkdir -p /home/lab6-pc18/hadoopdata/namenode

Step 9: Create DataNode directory

mkdir -p /home/lab6-pc18/hadoopdata/datanode

Step 10: Verify HDFS

configuration hdfs getconf -

confKey dfs.replication Expected:

1

Then:

hdfs getconf -confKey

dfs.namenode.name.dir Expected:

file:///home/lab6-pc18/hadoopdata/namenode

Then:

hdfs getconf -confKey

dfs.datanode.data.dir Expected:

file:///home/lab6-pc18/hadoopdata/datanode

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs getconf -confKey dfs.replication
2026-10-05 12:05:37,199 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
pplicable
1
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs getconf -confKey dfs.namenode.name.dir
2026-10-05 12:05:46,597 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
pplicable
file:///home/lab6-pc18/hadoopdata/namenode
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs getconf -confKey dfs.datanode.data.dir
2026-10-05 12:05:52,347 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
pplicable
```

MAPREDUCE CONFIGURATION

Step 11: Configure mapred-site.xml

nano $HADOOP_HOME/etc/hadoop/mapred-

site.xml Enter:

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Save and exit.

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ cat $HADOOP_HOME/etc/hadoop/mapred-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>

    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>

</configuration>
```

YARN CONFIGURATION

Step 12: Configure yarn-site.xml

nano $HADOOP_HOME/etc/hadoop/yarn-

site.xml Enter:

```
<?xml version="1.0" encoding="UTF-8"?>

<configuration>

  <property>

    <name>yarn.nodemanager.aux-services</name>

    <value>mapreduce_shuffle</value>

  </property>

  <property>

    <name>yarn.nodemanager.env-whitelist</name>


<value>JAVA_HOME,HADOOP_HOME,HADOOP_COMMON_HOME,HADOOP_COMMON_LIB
_NATIVE_D
IR,HADOOP_CONF_DIR,HADOOP_HDFS_HOME,HADOOP_MAPRED_HOME,HADOOP_YARN
_HOME</va
lue>

  </property>

</configuration>
```

Save and exit.

SSH CONFIGURATION

Step 13: Install SSH

sudo apt install openssh-server openssh-client -y

Step 14: Start SSH service

sudo systemctl start ssh

Step 15: Enable SSH at startup
sudo systemctl enable

ssh **Step 16: Check SSH**

status sudo systemctl

status ssh You should

see:

active

(running)

Press:

q

to exit the status screen.

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
lab6-pc18@lab6pc18-OptiPlex-7040:~$ ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
```

PASSWORDLESS SSH

Step 17: Generate SSH key

ssh-keygen -t rsa -P ""

When asked where to save the key, press **Enter**.

Add the public key to authorized keys

cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

Set permission for authorized

keys chmod 600

~/.ssh/authorized_keys

Step 18 : Test SSH

ssh

localhost

If asked:

Are you sure you want to continue

connecting? Type:

yes

Then

exit:

Exit

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
lab6-pc18@lab6pc18-OptiPlex-7040:~$ chmod 600 ~/.ssh/authorized_keys
lab6-pc18@lab6pc18-OptiPlex-7040:~$ ssh localhost
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
```

Step 19: Test passwordless SSH

ssh -o BtchMode=yes localhost echo

OK Expected:

OK

Step 20: Configure PDSH

export

PDSH_RCMD_TYPE=ssh

START HADOOP

Format the NameNode

Do this only for a new Hadoop installation.

hdfs namenode -format

Wait for the formatting to complete successfully.

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs namenode -format
WARNING: /home/lab6-pc18/hadoop/hadoop-3.3.6/logs does not exist. Creating.
2026-10-05 11:52:39,394 INFO namenode.NameNode: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting NameNode
STARTUP_MSG:   host = lab6pc18-OptiPlex-7040/127.0.1.1
```

Do not run this command again after the cluster is already working, because formatting creates a new HDFS namespace.

Step 21: Start HDFS

start-dfs.sh

Expected:

Starting namenodes on

[localhost] Starting datanodes

Starting secondary namenodes

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ start-dfs.sh
Starting namenodes on [lab6pc18-OptiPlex-7040]
pdsh@lab6pc18-OptiPlex-7040: lab6pc18-OptiPlex-7040: rcmd: socket: Permission denied
Starting datanodes
pdsh@lab6pc18-OptiPlex-7040: localhost: rcmd: socket: Permission denied
2026-10-05 11:52:51,093 ERROR conf.Configuration: error parsing conf core-site.xml
com.ctc.wstx.exc.WstxParsingException: Illegal processing instruction target ("xml")
 at [row,col,system-id]: [2,5,"file:/home/lab6-pc18/hadoop/hadoop-3.3.6/etc/hadoop/c
```

Step 22 : Check HDFS services

jps

Expected:

NameNo

de

DataNode

SecondaryNameNo

de Jps

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ jps
12288 SecondaryNameNode
11875 NameNode
12052 DataNode
12681 NodeManager
12538 ResourceManager
13039 Jps
```

Step 23 : Start YARN

start-yarn.sh

Expected:

Starting

resourcemanager

Starting

nodemanagers

WORDCOUNT APPLICATION

Step 24 : Create HDFS user directory

hdfs dfs -mkdir -p /user/$USER

Step 25: Create HDFS input directory

hdfs dfs -mkdir -p /user/$USER/input

Step 26: Create the WordCount input file

nano

~/wordcount.txt

Enter:

Hadoop is a framework

Hadoop is used for big

data

Hadoop supports distributed

processing Big data needs

distributed processing Save using:

Ctrl +O

Enter

Ctrl +

X

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -ls /
2026-10-05 12:08:54,213 WARN util.NativeCodeLoader: Unable to load native
pplicable
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -mkdir -p /user/$USER
2026-10-05 12:09:07,492 WARN util.NativeCodeLoader: Unable to load native
pplicable
lab6-pc18@lab6pc18-OptiPlex-7040:~$ nano ~/wordcount.txt
lab6-pc18@lab6pc18-OptiPlex-7040:~$ cat ~/wordcount.txt
Hadoop is a framework
Hadoop is used for big data
Hadoop supports distributed processing
Big data needs distributed processing
```

Check the input

file cat

~/wordcount.txt

Expected:

Hadoop is a framework

Hadoop is used for big

data

Hadoop supports distributed

processing Big data needs

distributed processing **Upload the**

file to HDFS

hdfs dfs -put ~/wordcount.txt /user/$USER/input/

Step 27:Verify the file in

HDFS hdfs dfs -ls

/user/$USER/input Expected

output will contain:

/user/lab6-pc18/input/wordcount.txt

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -mkdir -p /user/$USER/input
2026-10-05 12:09:35,850 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
pplicable
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -put ~/wordcount.txt /user/$USER/input/
2026-10-05 12:09:42,059 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
pplicable
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -ls /user/$USER/input
2026-10-05 12:09:50,563 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
```

Step 28: Run WordCount

hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount
/user/$USER/input

/user/$USER/output Wait until the

job completes successfully. hdfs dfs

-ls /user/$USER/

Output Expected:

part-r-00000

```
lab6-pc18@lab6pc18-OptiPlex-7040:~$ hdfs dfs -cat /user/$USER/output/part-r-00000
2026-10-05 12:10:20,760 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform..
```

Step 29: Display the final output

hdfs dfs -cat /user/$USER/output/part-r-00000

Final Output:

```
Big     1
Hadoop  3
a       1
big     1
data    2
distributed     2
for     1
framework       1
is      2
needs   1
processing      2
supports        1
used    1
```

RESULT:

The Hadoop 3.3.6 Single Node Cluster was successfully installed and configured on Ubuntu. HDFS and YARN services were started successfully. The WordCount MapReduce application was executed successfully, and the frequency of each word in the input file was calculated and displayed. Thus, the Hadoop Single Node Cluster and WordCount application were successfully implemented and executed.

Exp: 8	CREATING AND EXECUTING YOUR FIRST CONTAINER USING DOCKER

Aim

To install Docker, create a Docker container, execute the container, and verify its execution using Docker commands.

Software Requirements

- **Operating System: Ubuntu**
- **Docker**
- **Ubuntu Terminal**
- **Internet Connection**

Algorithm / Procedure

START

↓

Install Docker

↓

Start Docker Service

↓

Verify Docker Installation

↓

Download Ubuntu Docker Image

↓

Create and Run Docker Container

↓

Execute Commands Inside the Container

↓

Verify the Container

Stop and Remove the Container

↓

Step 1: Update the Package List

sudo apt update

Step 2: Install Docker

sudo apt install docker.io -y

```
renuk@RENUKA:~$ sudo apt install docker.io -y
Installing:
  docker.io
```

Step 3: Start Docker Service

sudo systemctl start docker

Step 4: Enable Docker Service

sudo systemctl enable docker

Step 5: Check Docker Version

docker –version

```
renuk@RENUKA:~$ sudo systemctl start docker
renuk@RENUKA:~$ sudo systemctl enable docker
renuk@RENUKA:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

Step 6: Check Docker Service

sudo systemctl status docker

The Docker service should show **active (running)**.

Step 7: Run the Hello World Container

sudo docker run hello-world

The message **Hello from Docker!** should be displayed.

```
renuk@RENUKA:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

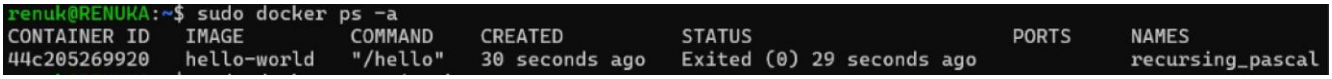
Step 8: Check Docker Images

sudo docker images



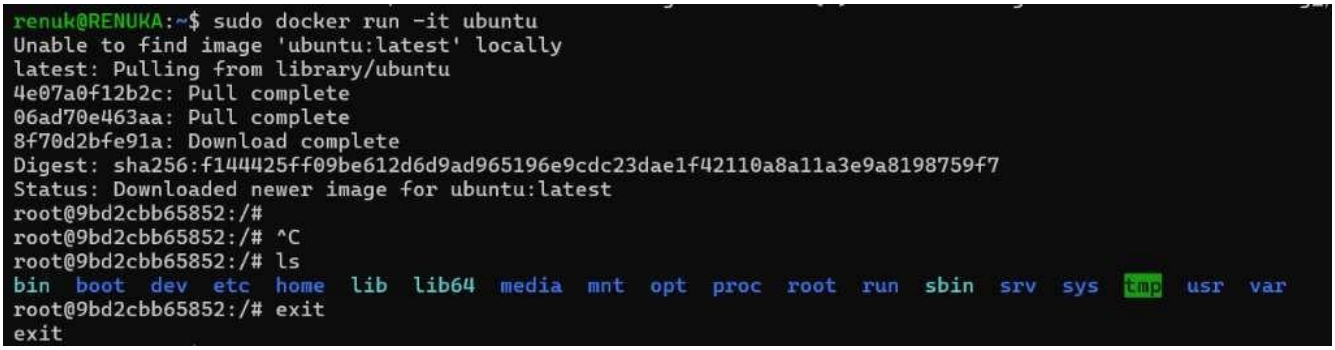
Step 9: Check Existing Containers

sudo docker ps -a



Step 10: Create and Run an Ubuntu Container

sudo docker run -it ubuntu



If the Ubuntu image is not available, Docker automatically downloads it.

Step 11: Check the Contents of the Container

Inside the container, execute:

ls

The Ubuntu directories such as bin, etc, home, usr, and var should be displayed.

Step 12: Exit from the Container

exit

Step 13: Verify the Container

sudo docker ps -a

```
renuk@RENUKA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED
9bd2cbb65852   ubuntu        "/bin/bash"   About a minute ago
44c205269920   hello-world   "/hello"      4 minutes ago
```

The Ubuntu container should be displayed with status **Exited (0)**.

Step 14: Start the Container Again

Your actual container name is pensive_dirac.

sudo docker start pensive_dirac

```
renuk@RENUKA:~$ sudo docker start pensive_dirac
[sudo: authenticate] Password:
pensive_dirac
```

Step 15: Check the Running Container

sudo docker ps

```
renuk@RENUKA:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED        STATUS          PORTS     NAMES
9bd2cbb65852   ubuntu    "/bin/bash"   25 hours ago   Up 12 seconds             pensive_dirac
```

The pensive_dirac container should be displayed with status **Up**.

Step 16: Execute a Command in the Running Container

sudo docker exec pensive_dirac echo "Docker Container is Running"

Expected output:

Docker Container is Running

```
renuk@RENUKA:~$ sudo docker exec pensive_dirac echo "Docker Container is Running"
Docker Container is Running
```

Step 17: Stop the Container

sudo docker stop pensive_dirac

Step 18: Remove the Container

sudo docker rm pensive_dirac

```
renuk@RENUKA:~$ sudo docker stop pensive_dirac
pensive_dirac
renuk@RENUKA:~$ sudo docker rm pensive_dirac
pensive_dirac
```

Step 19: Verify Container Removal
sudo docker ps -a

```
renuk@RENUKA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED        STATUS                      PORTS
61bc2d85dd80   ubuntu        "/bin/bash"   25 hours ago   Exited (0) 25 hours ago
44c205269920   hello-world   "/hello"      25 hours ago   Exited (0) 25 hours ago
```

The pensive_dirac container should no longer be displayed.

OUTPUT:

Docker version:

Docker version 29.1.3, build 29.1.3-0ubuntu4.1

Ubuntu container:

```
renuk@RENUKA:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

Status: Downloaded newer image for ubuntu:latest

root@9bd2cbb65852:/# ls

bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var

```
root@9bd2cbb65852:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@9bd2cbb65852:/# exit
```

root@9bd2cbb65852:/# exit

exit

After starting the container and executing the command:

Docker Container is Running

```
renuk@RENUKA:~$ sudo docker exec pensive_dirac echo "Docker Container is Running"
Docker Container is Running
```

RESULT:

Thus, Docker was successfully installed and configured. The Ubuntu Docker container was successfully created and executed, and its contents were verified. The container was also successfully started, stopped, and removed.

Exp: 9	RUN A CONTAINER FROM DOCKER HUB

Aim

To pull an image from Docker Hub and run a Docker container using the downloaded image.

Software Requirements

- **Operating System: Ubuntu**
- **Docker**
- **Docker Hub**
- **Internet Connection**

Algorithm / Procedure:

START
↓
Pull an image from Docker Hub
↓
Run the Docker container
↓
Verify the container
↓
Stop and remove the container
↓
STOP

Step 1: Check Docker Version

docker –version

```
renuk@RENUKA:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

Step 2: Pull the Image from Docker Hub

sudo docker pull hello-world

Step 3: Verify the Downloaded Image

sudo docker images

```
renuk@RENUKA:~$ sudo docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   5e2309035332       25.9kB         9.49kB   U
ubuntu:latest        f144425ff09b        162MB          45.6MB   U
```

The hello-world image should be displayed.

Step 4: Run the Docker Container

sudo docker run hello-world

```
renuk@RENUKA:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

The Docker Hub image is used to create and run the container.

Step 5: Check the Container

sudo docker ps -a

```
renuk@RENUKA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED          STATUS                      PORTS     NAMES
61bc2d85dd80   ubuntu        "/bin/bash"   42 seconds ago   Exited (0) 8 seconds ago              nice_lamport
9bd2cbb65852   ubuntu        "/bin/bash"   5 minutes ago    Exited (0) 4 minutes ago              pensive_dirac
44c205269920   hello-world   "/hello"      9 minutes ago    Exited (0) 9 minutes ago              recursing_pascal
```

The hello-world container should be displayed with status **Exited (0)**.

Step 6: Start the Container Again

For a hello-world container, restarting is not necessary because its program finishes immediately. Therefore, this step is not required.

Step 7: Remove the Container

First, find the container ID using:

sudo docker ps -a

```
renuk@RENUKA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED          STATUS                      PORTS     NAMES
61bc2d85dd80   ubuntu        "/bin/bash"   42 seconds ago   Exited (0) 8 seconds ago              nice_lamport
9bd2cbb65852   ubuntu        "/bin/bash"   5 minutes ago    Exited (0) 4 minutes ago              pensive_dirac
44c205269920   hello-world   "/hello"      9 minutes ago    Exited (0) 9 minutes ago              recursing_pascal
```

Then remove it using:

sudo docker rm <container_id>

Step 8: Verify Container Removal

sudo docker ps -a

The removed hello-world container should no longer be displayed.

OUTPUT:

```
renuk@RENUKA:~$ sudo docker pull hello-world
sudo docker run hello-world
[sudo: authenticate] Password:
Using default tag: latest
latest: Pulling from library/hello-world
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Image is up to date for hello-world:latest
docker.io/library/hello-world:latest

Hello from Docker!
```

RESULT:

Thus, Docker was successfully installed and configured. The Ubuntu Docker container was successfully created and executed, and its contents were verified. The container was also successfully started, stopped, and removed.

Exp: 10	CONTENT BEYOND SYLLABUS
	CREATE A KUBERNETES CLUSTER AND DEPLOY A SAMPLE APPLICATION

Aim:

To create a Kubernetes cluster using Minikube, deploy a sample application, expose it through a NodePort service, and access it using a web browser.

Software Requirements

- Windows with WSL2 and Ubuntu
- Docker
- Kubernetes
- Minikube
- Kubectl

Algorithm / Procedure:

Start

↓

Open the Ubuntu terminal

↓

Check Docker installation and status

↓

Start Minikube using the Docker driver

↓

Verify Minikube status

↓

Check Kubernetes nodes

↓

Create the hello-minikube deployment

↓

Verify deployment and pod status

↓

Expose the deployment using a NodePort service

↓

Check the service status

↓

Access the application through a web browser

↓

Verify the application output

↓

Stop

Steps :

Step 1: Check Docker

docker –version

```
renuk@RENUKA:~$ docker --version
docker ps
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

docker ps

Check that Docker is installed and accessible.

Step 2: Start Minikube

minikube start --driver=docker

```
renuk@RENUKA:~$ minikube start --driver=docker
😄  minikube v1.39.0 on Ubuntu 26.04 (kvm/amd64)
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.51 ...
```

This starts a local Kubernetes cluster using Docker as the driver.

Step 3: Check Minikube status

minikube status

```
renuk@RENUKA:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Verify that the Minikube components are running.

Step 4: Check Kubernetes nodes

kubectl get nodes

```
renuk@RENUKA:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE     VERSION
minikube   Ready    control-plane   3d10h   v1.37.0
```

The node should show the status Ready.

Step 5: Create the sample deployment

kubectl create deployment hello-minikube --image=kicbase/echo-server:1.0

This creates a deployment named hello-minikube. If it already exists, skip this command and continue.

Step 6: Check the deployment

kubectl get deployments

```
renuk@RENUKA:~$ kubectl get deployments
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
hello-minikube   1/1     1            1           3d10h
```

Wait until the deployment shows one available replica.

Step 7: Check the application pod

kubectl get pods

```
renuk@RENUKA:~$ kubectl get pods
NAME                              READY   STATUS    RESTARTS      AGE
hello-minikube-5658b68679-scrv6   1/1     Running   1 (10m ago)   3d10h
```

Wait until the application pod shows 1/1 under READY and Running under STATUS.

Step 8: Expose the application

kubectl expose deployment hello-minikube --type=NodePort --port=8080

This creates a NodePort service. If the service already exists, skip this command.

Step 9: Check the service

kubectl get services

```
renuk@RENUKA:~$ kubectl get services
NAME             TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
hello-minikube   NodePort    10.99.231.2   <none>        8080:31568/TCP   3d9h
kubernetes       ClusterIP   10.96.0.1     <none>        443/TCP          3d10h
```

Confirm that hello-minikube appears in the service list.

Step 10: Open the application

minikube service hello-minikube

```
renuk@RENUKA:~$ minikube service hello-minikube
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube	8080	http://192.168.49.2:31568

```
🔗  Starting tunnel for service hello-minikube.
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube		http://127.0.0.1:34813

```
🎉  Opening service default/hello-minikube in default browser...
```

Open the exact URL shown in the terminal in your browser. Keep the terminal open if Minikube starts a tunnel.

Step 11: Verify deployment rollout

kubectl rollout status deployment/hello-minikube --timeout=180s

Expected message: deployment "hello-minikube" successfully rolled out.

Output:

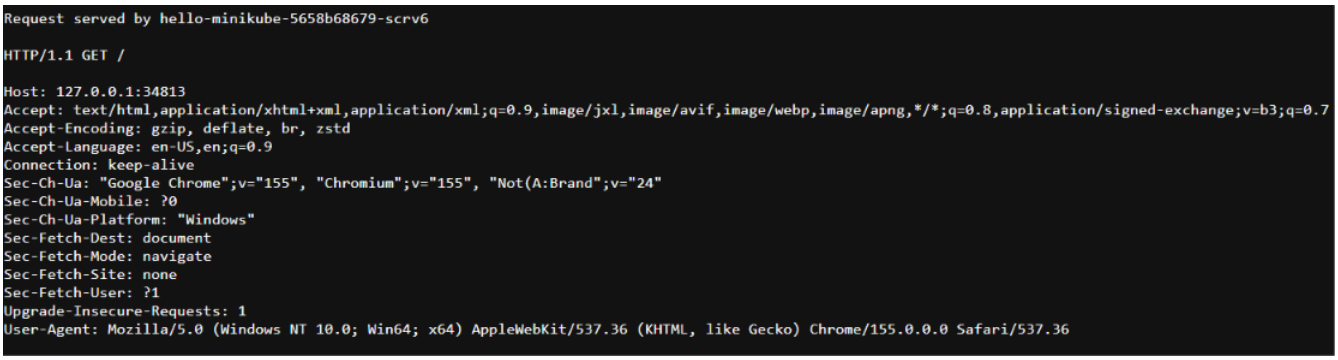
The browser displayed the following response from the deployed application:

Request served by hello-minikube-5658b68679-scrv6

HTTP/1.1 GET /

Host: 127.0.0.1:34813

```
🎉  Opening service default/hello-minikube in default browser...
👉  http://127.0.0.1:34813
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```



Result:

Thus, the Kubernetes cluster was successfully created using Minikube, and the hello-minikube sample application was deployed and exposed through a NodePort service. The application was accessed successfully through a web browser.

Exp: 11	IMPLEMENT REST API SERVICES AND ACCESS THEM FROM A CLOUD-HOSTED APPLICATION

Aim:

To implement REST API services using Python Flask, test GET and POST requests, and access the API through a web browser.

Software Requirements

- Ubuntu Linux / Windows with WSL2
- Python 3
- Flask
- Nano text editor
- Google Chrome
- cURL

Algorithm / Procedure:

Start

↓

Open the Ubuntu terminal

↓

Check Python installation

↓

Create a project directory

↓

Create app.py using the Nano editor

↓

Write the Flask REST API code

↓

Create requirements.txt

↓

Install Flask and Gunicorn

↓

Run the Flask application

↓

Implement and test the GET API

↓

Implement and test the POST API

↓

Access the API using a web browser

↓

Deploy the application to a cloud hosting platform

↓

Verify the API response

↓

Stop

Steps :

Step 1: Check Python installation

python3 –version

Step 2: Create the project directory

mkdir -p FlaskRESTAPI

cd FlaskRESTAPI

```
renuk@RENUKA:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
renuk@RENUKA:~$ python3 --version
Python 3.14.4
renuk@RENUKA:~$ pip install flask
```

Step 3: Create the Python file
nano app.py

Enter the following code:

from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route("/")

```
def home():
    return "REST API is running successfully!"
@app.route("/api/student", methods=["GET"])
def get_student():
    student = {
        "id": 1,
        "name": "Renuka",
        "course": "Computer Science"
    }
    return jsonify(student)
@app.route("/api/student", methods=["POST"])
def add_student():
    data = request.get_json()
    return jsonify({
        "message": "Student data received successfully",
        "student": data
    }), 201
 if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Save and exit Nano:

- Press Ctrl + O.
- Press Enter.
- Press Ctrl + X.

```
  GNU nano 8.7.1                                          app.py
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route("/")
def home():
    return "REST API is running successfully!"

@app.route("/api/student", methods=["GET"])
def get_student():
    student = {
        "id": 1,
        "name": "Renuka",
        "course": "Computer Science"
    }
    return jsonify(student)

@app.route("/api/student", methods=["POST"])
def add_student():
    data = request.get_json()
    return jsonify({
        "message": "Student data received successfully",
        "student": data
    }), 201

if __name__ == "__main__":
```

Step 4: Create the requirements file

nano requirements.txt

Enter:

Flask

Gunicorn

```
  GNU nano 8.7.1                                   requirements.txt
Flask
gunicorn
```

Save using Ctrl + O, press Enter, and exit using Ctrl + X.

Step 5: Install the required packages

Create a virtual environment and activate it:

python3 -m venv .venv

source .venv/bin/activate

Install the packages:

pip install -r requirements.txt

```
renuk@RENUKA:~/FlaskRESTAPI$ sudo apt update
sudo apt install python3-venv -y
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
```

Step 6: Run the Flask application

python3 app.py

Expected terminal message:

* Running on http://127.0.0.1:5000

```
(.venv) renuk@RENUKA:~/FlaskRESTAPI$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.38.92:5000
```

Keep this terminal open while testing the API.

Step 7: Test the GET API

Open a second Ubuntu terminal. If you created a virtual environment, activate it if needed, then run:

curl http://127.0.0.1:5000/api/student

Output:

{"course":"Computer Science","id":1,"name":"Renuka"}

The GET method retrieves student information from the API.

Step 8: Test the POST API

Run:

```
curl -X POST http://127.0.0.1:5000/api/student \
-H "Content-Type: application/json" \
-d '{"id":2,"name":"Anu","course":"Computer Science"}'
```

Expected output:


```
 {
   "message": "Student data received successfully",
   "student": {
     "course": "Computer Science",
     "id": 2,
     "name": "Anu"
   }
 }
```


The POST method receives JSON data and returns a confirmation response. This example does not permanently store the student record.

Step 9: Access the API through a browser

Open Google Chrome and enter:

http://127.0.0.1:5000/api/student

The browser displays:


```
 {
   "course": "Computer Science",
   "id": 1,
```

"name": "Renuka"

}

```
renuk@RENUKA:~$ curl http://127.0.0.1:5000/api/student
{
  "course": "Computer Science",
  "id": 1,
  "name": "Renuka"
}
renuk@RENUKA:~$ curl -X POST http://127.0.0.1:5000/api/student \
-H "Content-Type: application/json" \
-d '{"id":2,"name":"Anu","course":"Computer Science"}'
{
  "message": "Student data received successfully",
  "student": {
    "course": "Computer Science",
    "id": 2,
    "name": "Anu"
  }
}
```

Step 10: Deploy the application to the cloud

To access the API from a public cloud URL:

1. Upload app.py and requirements.txt to a Git repository.
2. Create a Python web service on a compatible cloud hosting platform.
3. Configure the build or installation command:

pip install -r requirements.txt

4. Configure the start command:

gunicorn app:app

5. Deploy the application and copy the public URL provided by the hosting service.
6. Open the following endpoint using your actual cloud URL:

https://your-cloud-url/api/student

The cloud endpoint will return the same student JSON response if deployment succeeds. Cloud deployment must be completed separately; the local test alone does not mean the application is cloud-hosted.

Output:

GET API output

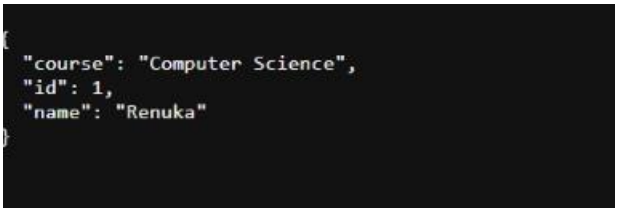
{"course":"Computer Science","id":1,"name":"Renuka"}

POST API output


```
{
  "message": "Student data received successfully",
  "student": {
    "id": 2,
    "name": "Anu",
    "course": "Computer Science"
  }
}
```


Browser output

http://127.0.0.1:5000/api/student



The browser displays the student details in JSON format.

Result

Thus, REST API services were implemented successfully using Python Flask. The GET and POST endpoints were tested using cURL, and the API was accessed through a web browser. The application can also be accessed publicly after successful deployment to a cloud hosting platform.